КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики

Звіт до лабораторної роботи №4
за спеціальністю 122 Комп'ютерні науки
на тему:

ОБРОБКА ВІДЕО НА GPU ІЗ CUDA

Виконав студент
Божевський Іван

КИЇВ 2026

РЕФЕРАТ

У роботі реалізовано конвеєр обробки відео засобами OpenCV і CUDA. Відеофайл декодується послідовно, поточний та попередній кадри передаються до пам'яті GPU, для них обчислюється абсолютна попарна різниця, а результат записується до нового MP4-файлу. Програма не має CPU fallback і потребує OpenCV, скомпільованого з підтримкою CUDA.

Ключові слова: CUDA, GPU, OPENCV, ОБРОБКА ВІДЕО, ПОПАРНА РІЗНИЦЯ КАДРІВ

ЗМІСТ

РЕФЕРАТ 2
ВСТУП 4
1 Теоретичні відомості та алгоритм 5
2 Реалізація програми 6
3 Вхідні дані та середовище 7
4 Методика запуску та результати 8
ВИСНОВКИ 9

ВСТУП

Метою роботи є реалізація обробки відео на графічному процесорі. Завдання включає відкриття відеофайлу за допомогою OpenCV, завантаження кадрів у пам'ять GPU, обчислення попарної різниці кадрів та збереження результуючого відео засобами OpenCV. Така операція є базою для виявлення руху, фоновіднімання та попередньої обробки відеопотоку.

1 ТЕОРЕТИЧНІ ВІДОМОСТІ ТА АЛГОРИТМ

Нехай $I_t(x, y, c)$ --- значення каналу c пікселя з координатами (x, y) у кадрі t. Результат попарної різниці визначається незалежно для кожного пікселя й каналу:

$$D_t(x, y, c) = |I_t(x, y, c) - I_{t-1}(x, y, c)|, \qquad t \geq 1. \tag{1.1}$$

Для першого кадру D_0 записується чорним, оскільки попереднього кадру немає. Операція має часову складність $O(W \cdot H \cdot C)$ на кадр, але її пікселі не залежать один від одного, тому вона природно відображається на GPU.

CUDA --- платформа NVIDIA для виконання паралельних обчислень на GPU. У OpenCV клас `cv::cuda::GpuMat` представляє матрицю в пам'яті GPU. Метод `upload()` копіює декодований `cv::Mat` з оперативної пам'яті на пристрій, `cv::cuda::absdiff()` виконує різницю на GPU, а `download()` повертає результат у `cv::Mat` для `cv::VideoWriter`. Декодування й кодування лишаються в OpenCV на CPU, тому вимірюваний час повного конвеєра також містить передавання даних CPU--GPU.

2 РЕАЛІЗАЦІЯ ПРОГРАМИ

Програму реалізовано мовою C++17 у файлі `src/cuda_video_difference.cpp`. CMake знаходить модулі OpenCV `core`, `videoio`, `imgproc` та `cudaarithm`; відсутність CUDA-модуля є помилкою конфігурації, а не непомітним переходом на CPU.

Алгоритм має такі етапи:

1. `cv::VideoCapture` відкриває вхідний MP4 і зчитує його параметри: ширину, висоту та частоту кадрів.
2. Перший кадр завантажується в `previous_gpu`; до результуючого відео записується чорний кадр такого самого розміру.
3. Для кожного наступного кадру він завантажується у `current_gpu`; виклик `cv::cuda::absdiff(current_gpu, previous_gpu, difference_gpu)` виконує обчислення на GPU.
4. `difference_gpu.download()` повертає різницю в RAM, після чого `cv::VideoWriter` кодує її кодеком `mp4v`. Поточний кадр стає попереднім для наступної ітерації.

Перед початком обробки програма перевіряє число пристроїв методом `getCudaEnabledDeviceCount` модуля `cv::cuda`. За відсутності CUDA-пристрою або CUDA-збірки OpenCV вона завершується з пояснювальним повідомленням і не видає CPU-результат за GPU-обробку.

3 ВХІДНІ ДАНІ ТА СЕРЕДОВИЩЕ

Використано файл `16756162_1280_720_25fps.mp4` розміром 7 016 820 байт, що містить 291 кадр роздільністю 1280×720 із частотою 25 кадрів/с. Локальна машина має NVIDIA GeForce RTX 4070 та драйвер версії 621.01. Для запуску встановлено CUDA Toolkit 13.5 і OpenCV 4.14, зібраний vcpkg з модулями `cudaarithm` та FFmpeg. Звичайний пакет `opencv-python` із PyPI не застосовується, оскільки він не містить CUDA-модулів.

4 МЕТОДИКА ЗАПУСКУ ТА РЕЗУЛЬТАТИ

Програму збирають командами `cmake -S . -B build` і `cmake --build build`, після чого запускають із двома аргументами: шляхом до вхідного та вихідного MP4. Перед відкриттям відео виконується перевірка наявності CUDA-пристрою. Якщо OpenCV не має CUDA-підтримки або GPU недоступний, програма завершується з помилкою; реалізація не виконує обчислення різниці на CPU.

Під час одного локального запуску на RTX 4070 створено результуюче відео розміром 3 409 817 байт. Виміряний час повного конвеєра становить 1.447 с; він включає запуск програми, декодування, копіювання CPU--GPU, CUDA-обчислення, повернення кадру в RAM і кодування MP4. Пропускна здатність становить $291/1.447 = 201.106$ кадрів/с.

Таблиця 4.1 --- Результат локального GPU-запуску

Параметр	Значення
GPU	NVIDIA GeForce RTX 4070
CUDA Toolkit	13.5
OpenCV	4.14 з `cudaarithm`
Кадрів	291
Час повного конвеєра	1.447 с
Пропускна здатність	201.106 FPS
Вихідний файл	`frame_difference.mp4`

Різницевий кадр містить світлі області лише там, де значення пікселів змінилися між двома послідовними кадрами. Статичні ділянки наближаються до чорного, а рухомі об'єкти та зміни освітлення стають помітними.

ВИСНОВКИ

Реалізовано програму CUDA/OpenCV для попарної різниці послідовних кадрів відео. Вона відкриває MP4 через `VideoCapture`, передає кадри у `cv::cuda::GpuMat`, обчислює `cv::cuda::absdiff` на GPU та записує результат через `VideoWriter`. CPU fallback навмисно відсутній: за відсутності CUDA-пристрою або CUDA-збірки OpenCV програма завершується з помилкою. На NVIDIA GeForce RTX 4070 оброблено 291 кадр 1280×720 за 1.447 с, тобто з пропускною здатністю 201.106 FPS; створено результатний файл `frame_difference.mp4`.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

[1] OpenCV Documentation. CUDA-accelerated Computer Vision. https://docs.opencv.org/4.x/d2/dbc/cuda_intro.html.

[2] OpenCV Documentation. Video I/O module. https://docs.opencv.org/4.x/dd/de7/group_videoio.html.

[3] NVIDIA. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.